

Sistemas de Big Data

Actividad Fase 2:



Curso de Especialización en Inteligencia Artificial y Big Data

MODALIDAD ONLINE

Grupo 9:

- Alejandro Fernández Fuentes - Data
- Antoni Sureda Cardona - Platform
- Blas Martos Ortega - Machine Learning
- Hugo Barrera Castillo - Project Management / BI

Índice

1. Estado actual del producto.....	2
2. Guía de ejecución local.....	3
3. Evidencias funcionales.....	4
4. Arquitectura real implementada.....	6
5. Índice de código (estructura del proyecto).....	7
6. Código completo del proyecto.....	10
7. Breve explicación por módulo.....	10
7.1. Preparación y limpieza de los datos.....	10
7.2. Importación de los datos en PostgreSQL.....	10
7.3. Frontend.....	11
7.4 Pipeline.....	12
7.5 Database.....	13
7.6 Orquestador.....	13
7.7 Backend.....	13
7.8. Preparación del dataset para Machine Learning.....	14
7.8.1 Construcción del dataset.....	15
7.8.2 Preprocesado de los datos.....	15
7.9. Entrenamiento del modelo de Machine Learning.....	15
7.9.1 Características del modelo.....	15
7.9.2 Generación de predicciones futuras.....	16
7.9.3. Exportación de las predicciones.....	16
7.9.4 Inserción automática de predicciones en PostgreSQL.....	16
7.9.5. Conclusión.....	16
8. Limitaciones actuales.....	17
8.1 Bugs y problemas conocidos.....	17
8.2 Atajos técnicos asumidos.....	17
8.3 Partes incompletas o no implementadas.....	17
8.4 Consideración final sobre las limitaciones.....	17
9. Repositorio obligatorio.....	17

1. Estado actual del producto

Qué funcionalidades están completas:

- **Arquitectura y Contenerización:** Despliegue íntegro del ecosistema mediante Docker, orquestando cuatro microservicios independientes y comunicados entre sí (Base de Datos, Backend, Frontend y Pipeline).
- **ETL:** Pipeline automatizado en Python que extrae el histórico, depura los registros y realiza una sincronización incremental eficiente contra la base de datos PostgreSQL.
- **Backend:** Desarrollo de una API REST utilizando FastAPI que expone las métricas mediante endpoints, implementando un patrón de repositorios y tipado estricto.
- **Frontend:** Despliegue de un Dashboard visual e interactivo servido a través de Nginx, que consume correctamente el JSON de la API para mostrar gráficos e indicadores de negocio.

Qué partes funcionan parcialmente:

- **Modelo predictivo de ML:** El algoritmo se inicializa, procesa el histórico, completa su entrenamiento y persiste las predicciones en la base de datos sin errores de ejecución. No obstante, a nivel matemático está calculando previsiones excesivamente optimistas (sobredimensionadas respecto a la media semanal), lo que requiere una futura revisión analítica.

Qué queda pendiente para Fase 3:

- **Fine Tuning Modelo Predictivo:** Revisión de la sensibilidad del algoritmo frente a la estacionalidad y el contexto climático para corregir el sesgo alcista de las predicciones.
- **Alertas de Stock de Seguridad:** Integración de un sistema de notificaciones en el panel de control para avisar al responsable de compras sobre posibles roturas de inventario o exceso de perecederos.
- **Módulo de Exportación de Pedidos:** Funcionalidad para consolidar las predicciones finales y exportarlas en formatos compatibles para el envío directo a proveedores

2. Guía de ejecución local

Requisitos previos Para garantizar una ejecución limpia y sin conflictos de dependencias, la máquina anfitriona debe contar con el siguiente software instalado:

- **Git:** Para la clonación del repositorio remoto.
- **Docker:** Motor de contenedores (versión 20.10 o superior).
- **Docker Compose:** Herramienta de orquestación de contenedores (versión V2 recomendada).

1. Clonar el repositorio

Abre la terminal en tu máquina anfitriona y descarga el código fuente:

- `git clone https://github.com/ToniSureda/smartchef.git`
- `cd smartchef`

2. Levantar los contenedores:

- `docker exec -it smartchef_pipeline bash run_pipeline.sh`

3. Ejecutar el pipeline (primera carga y predicción) Dado que el contenedor del orquestador está programado para ejecutarse de madrugada mediante *cron*, forzamos la primera ejecución manual del ETL para poblar el sistema y generar las predicciones:

`docker exec -it smartchef_pipeline bash run_pipeline.sh`

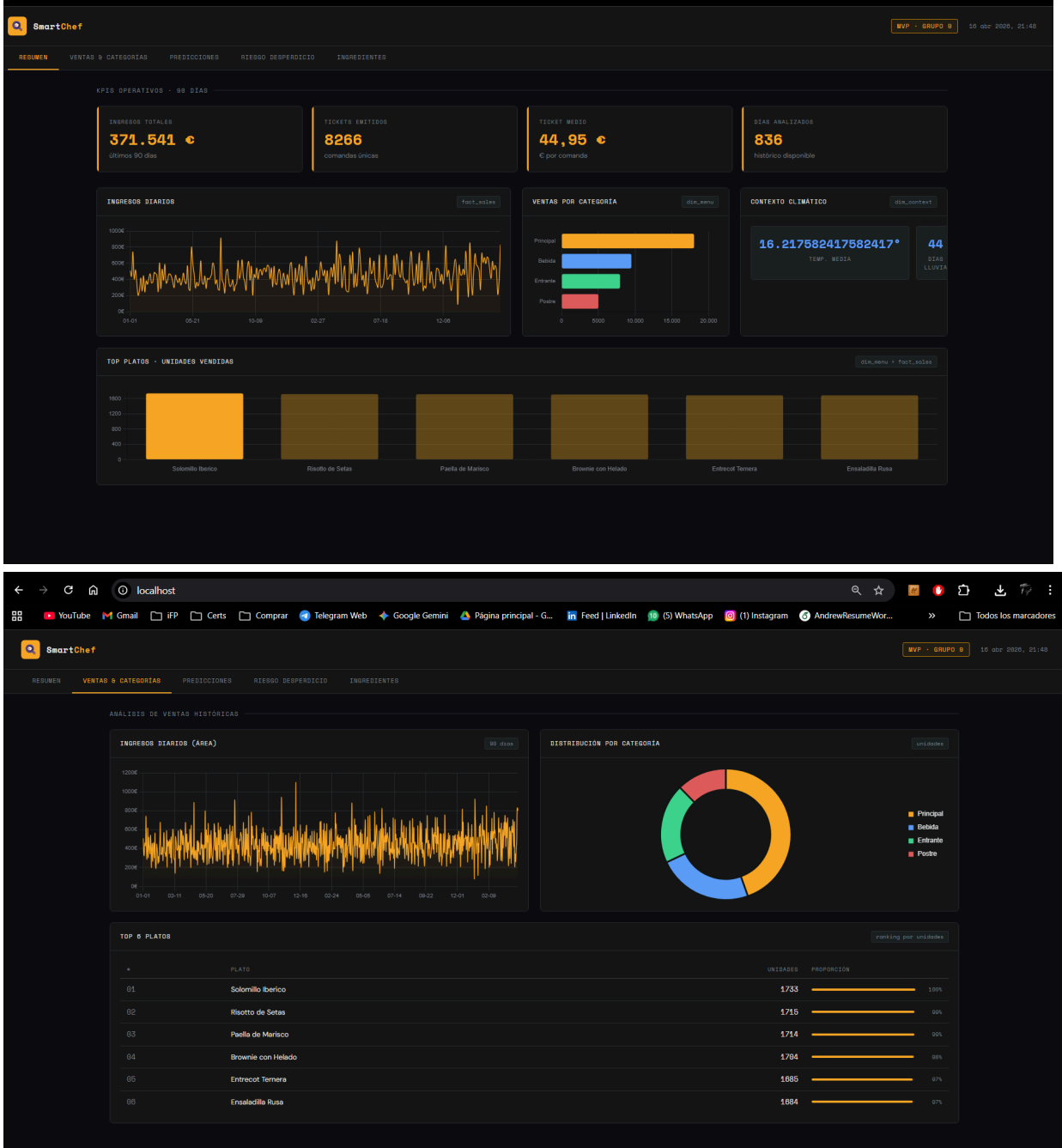
4. Acceder al sistema Una vez finalizado el script del pipeline, el producto estará 100% operativo en las siguientes rutas locales:

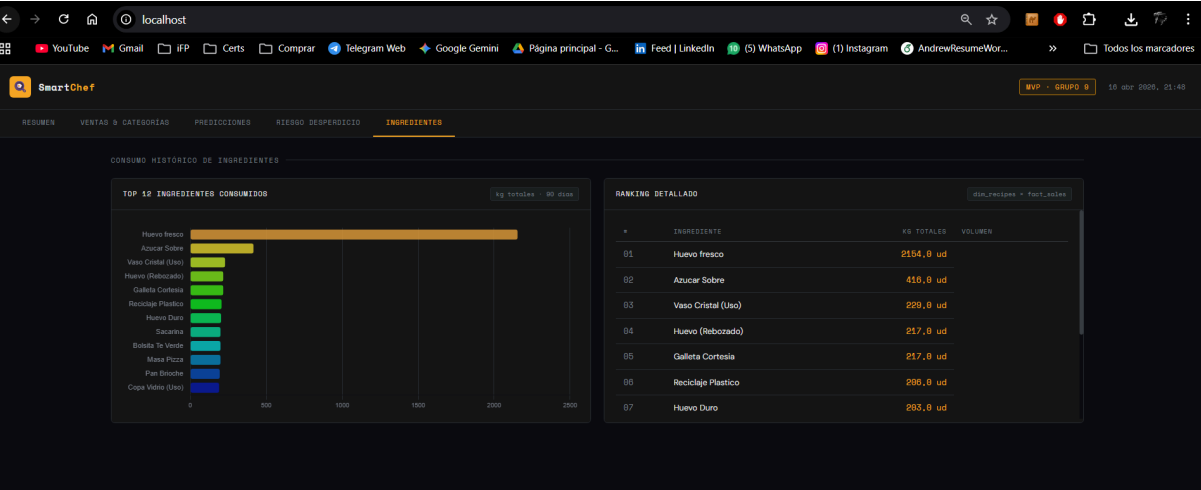
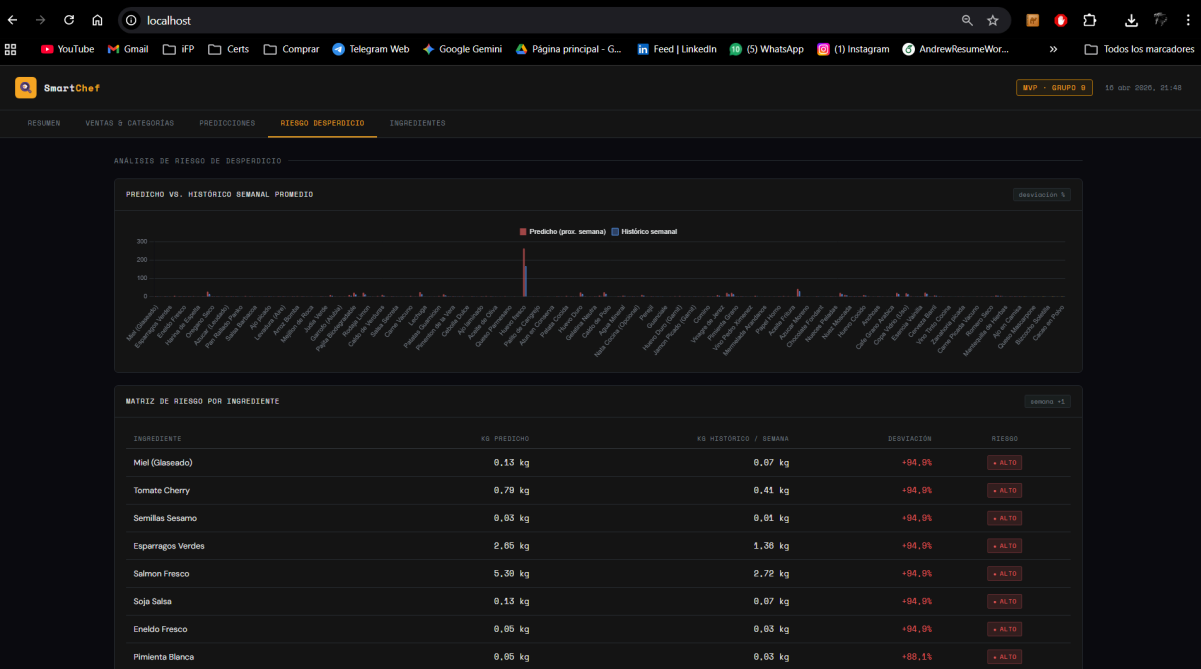
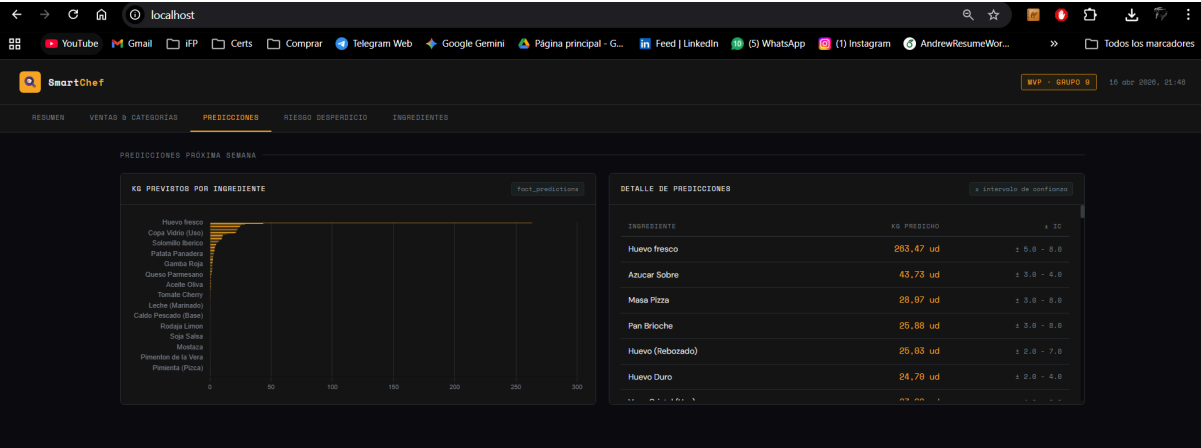
Dashboard: `http://localhost`

API Docs: `http://localhost:8000/docs`

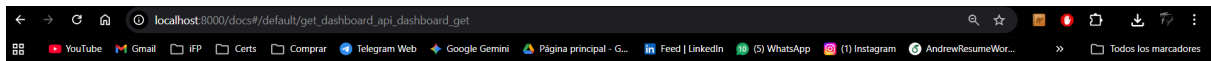
3. Evidencias funcionales

Capturas:
Front funcionando





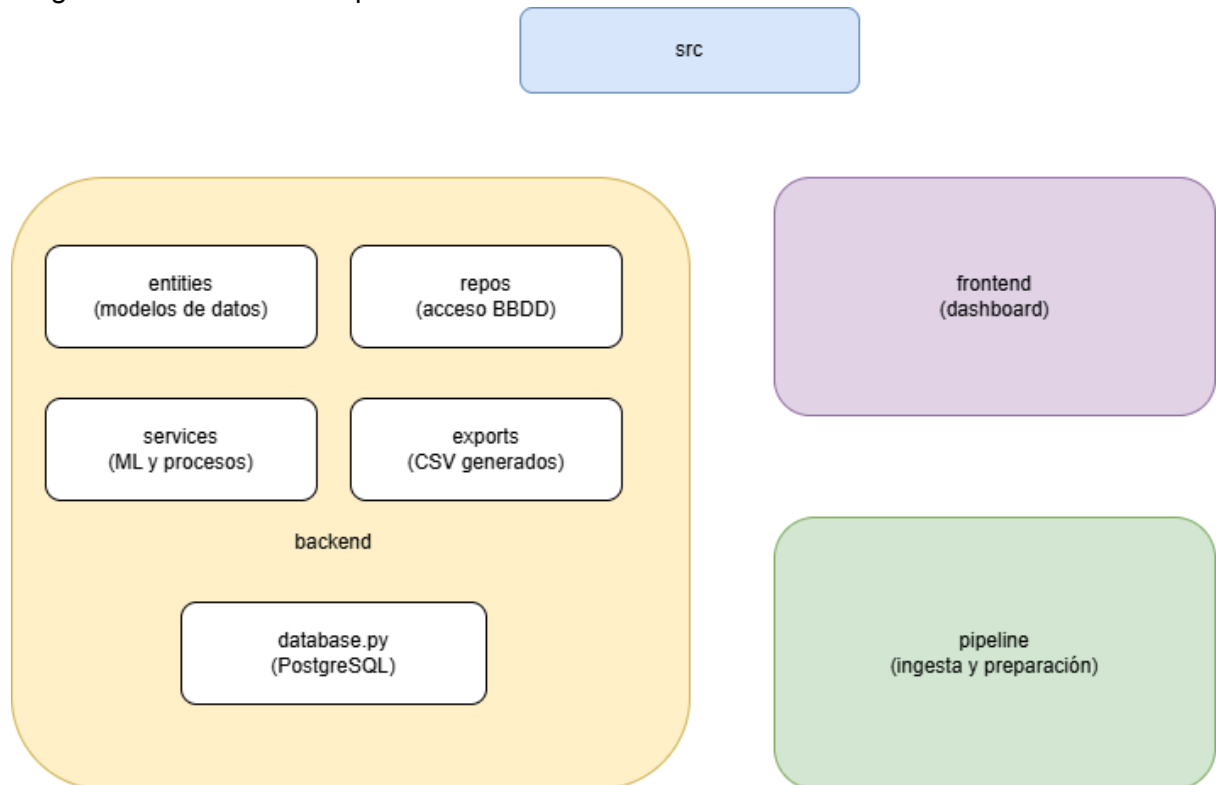
- Backend activo



```
asureda@toniPortatil:/mnt/c/Users/tonis/Documents/TFM_ProyectoFinal/smartchef$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS
625e99e80e3f   smartchef-pipeline    "/bin/sh -c 'printen..." 16 minutes ago Up 16 minutes
71d5ae1b35fe   smartchef-backend     "uvicorn main:app --..." 16 minutes ago Up 16 minutes   0.0.0.0:8000->80
00/tcp, [::]:8000->8000/tcp
1dc25c077c26   smartchef-db          "docker-entrypoint.s..." 16 minutes ago Up 16 minutes (healthy) 0.0.0.0:5432->54
32/tcp, [::]:5432->5432/tcp
4da0b5258c78   nginx:alpine          "/docker-entrypoint...." 16 minutes ago Up 16 minutes   0.0.0.0:80->80/t
cp, [::]:80->80/tcp
asureda@toniPortatil:/mnt/c/Users/tonis/Documents/TFM_ProyectoFinal/smartchef$ |
```

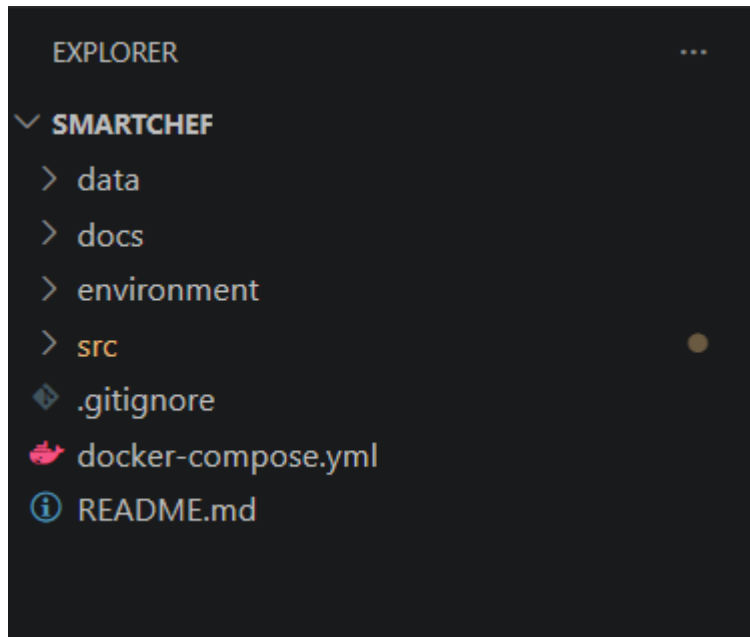
4. Arquitectura real implementada

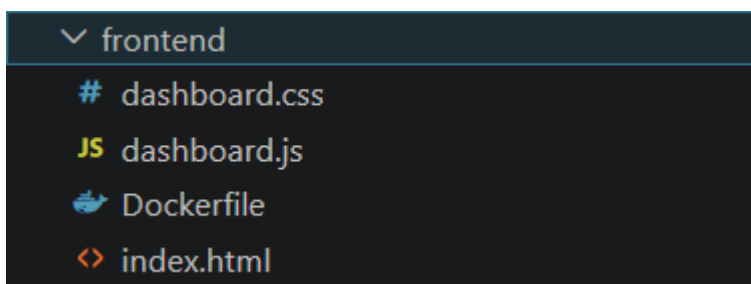
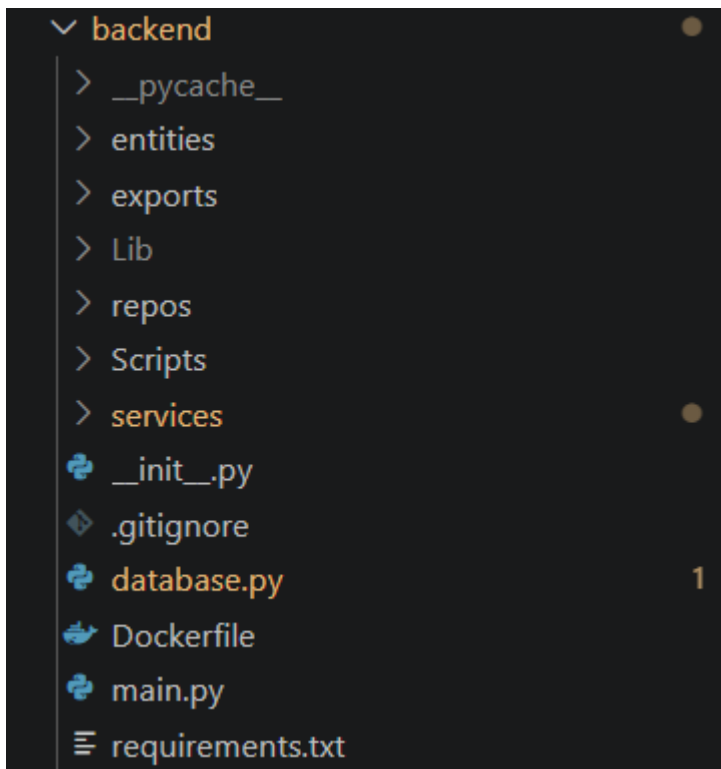
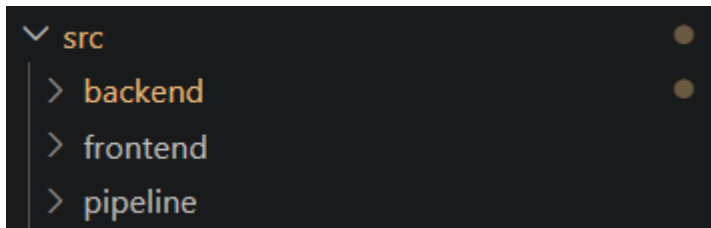
Diagrama actualizado + explicación.



5. Índice de código (estructura del proyecto)

Debe incluir la estructura REAL del repositorio, por ejemplo:





▼ pipeline

🔗 __init__.py

🔗 context_init.py

🔗 context_update.py

🔗 db_sync.py

🐳 Dockerfile

🔗 generate_raw_historic.py

🔗 ingest.py

≡ requirements.txt

💰 run_pipeline.sh

🔒 .gitkeep

6. Código completo del proyecto

<https://github.com/ToniSureda/smartchef>

7. Breve explicación por módulo

7.1. Preparación y limpieza de los datos

El primer paso del proyecto consistió en la **preparación y limpieza de los datos de entrada**, que posteriormente serían utilizados tanto para la base de datos como para el modelo de Machine Learning.

Se trabajó con los siguientes ficheros CSV:

- `maestro_platos_limpio.csv`: información general de los platos.
- `recetas_ingredientes_limpio.csv`: ingredientes y costes asociados a cada plato.
- `ventas_historico_limpio.csv`: histórico de ventas.
- `dim_context.csv`: información contextual (festivos, días de la semana y variables meteorológicas).

Estos archivos fueron previamente limpiados para asegurar:

- formatos de fecha correctos,
- ausencia de valores inconsistentes,
- coherencia entre identificadores (`id_plato`).

7.2. Importación de los datos en PostgreSQL

Los datos de los CSV se importaron en PostgreSQL siguiendo un **orden lógico**:

1. `dim_menu`
2. `dim_recipes`
3. `dim_context`
4. `fact_sales`

Durante la importación se tuvieron en cuenta:

- la codificación de caracteres (`latin-1`),
- las claves foráneas,

- y la consistencia entre fechas y platos.

7.3. Frontend

[Acceso al código del frontend](#)

index.html:

Es el documento base de la aplicación web. Define la arquitectura semántica de la interfaz, estructurando la navegación por pestañas (Resumen, Ventas, Predicciones, Riesgo y Desperdicio). Además, se encarga de importar las librerías de terceros (como Chart.js para las gráficas y las tipografías de Google Fonts) y preparar los contenedores vacíos (etiquetas `<canvas>` y `<table>`) donde el código dinámico inyectará la información.

dashboard.css:

Es la hoja de estilos maestra. Está desarrollada utilizando variables CSS (`:root`) para implementar un sistema de diseño estandarizado en "modo oscuro", optimizando el contraste visual. Utiliza técnicas modernas de maquetación (Flexbox y CSS Grid) para distribuir los paneles informativos y contiene reglas adaptativas (media queries) para asegurar que el panel se redimensione correctamente en pantallas más pequeñas.

dashboard.js:

Actúa como el motor interactivo del frontend. Su función principal es ejecutar peticiones HTTP asíncronas (`fetch`) hacia el endpoint expuesto por tu backend en FastAPI. Una vez recibe el JSON con las métricas históricas y las predicciones de IA, procesa estos datos y utiliza Chart.js para dibujar dinámicamente las gráficas, rellenar las tablas y gestionar los clics del usuario en el menú de navegación.

Dockerfile:

Define la infraestructura necesaria para desplegar este frontend en cualquier entorno. Utiliza una imagen base ultraligera de Nginx sobre Alpine Linux (`nginx:alpine`). El script limpia los archivos por defecto del servidor, copia tu HTML, CSS y JS a la ruta pública de Nginx (`/usr/share/nginx/html/`), aplica los permisos de seguridad necesarios y expone el puerto 80 para servir la interfaz web de manera autónoma y eficiente

7.4 Pipeline

[Acceso a pipeline](#)

generate_raw_historic.py: Se encarga de la **generación de datos históricos de ventas en bruto** (raw).

A partir del maestro de platos limpio, simula ventas diarias introduciendo errores controlados (cantidades negativas o fuera de rango), reproduciendo así las imperfecciones típicas de datos procedentes de sistemas operativos reales.

ingest.py: Implementa el proceso de **limpieza y validación de los datos de ventas**:

- Filtra registros anómalos o ilógicos.
- Garantiza la integridad referencial cruzando con el maestro de platos.
- Produce un dataset de ventas limpio, consistente y listo para su explotación analítica.

context_init.py: Genera la **dimensión de contexto temporal**, creando un calendario enriquecido con variables externas:

- Festivos y vísperas.
- Día de la semana.
- Estructura preparada para datos meteorológicos. Este dataset sirve como apoyo para el análisis y modelado posterior.

context_update.py

Permite mantener actualizada la dimensión de contexto mediante **actualizaciones incrementales**, añadiendo únicamente nuevas fechas y evitando duplicidades, asegurando continuidad temporal y consistencia del dato.

dockerfile: Define el entorno de ejecución del proyecto mediante Docker, instalando Python, las dependencias necesarias y el servicio cron. Permite automatizar la ejecución diaria del proceso a las 03:00 AM dentro de un contenedor reproducible, garantizando consistencia del entorno y registro de logs sin intervención manual.

run_pipeline.sh: Script de shell encargado de **orquestrar la ejecución secuencial del sistema**, lanzando los distintos módulos del proyecto. Actualiza los datos, sincroniza la base de datos, entrena el modelo predictivo y carga las predicciones finales, asegurando que todo el flujo se ejecute de forma controlada y automatizada.

7.5 Database

[Acceso al código de Database](#)

Dockerfile: Es el archivo de configuración que construye la imagen de la base de datos basándose en PostgreSQL 17. Su función principal es copiar el script de inicialización en una carpeta especial de Postgres (/docker-entrypoint-initdb.d/) para que se ejecute automáticamente al arrancar el contenedor por primera vez, le otorga los permisos necesarios y mueve el archivo de volcado SQL a una carpeta temporal (/tmp/) para tenerlo listo.

init-db.sh: Es un script ejecutable de Bash que actúa como automatizador. Una vez que el contenedor de PostgreSQL se levanta, este script entra en acción ejecutando el comando pg_restore. Su trabajo es inyectar toda la información del archivo .sql dentro de la base de datos recién creada, ignorando propietarios y privilegios antiguos (--no-owner --no-privileges) para evitar conflictos de permisos en el nuevo entorno de Docker.

SmartChefBBDD.sql: Es el archivo de volcado de la base de datos. Contiene toda la estructura relacional de tu proyecto (tablas de ventas, menú, contexto, recetas, etc.) y los datos iniciales necesarios para que, al levantar el sistema, el backend y el modelo de Machine Learning tengan información con la que trabajar desde el segundo cero.

7.6 Orquestador

[Acceso al código del Orquestador](#)

docker-compose.yml: Es el archivo de orquestación central que define y levanta los cuatro microservicios del ecosistema (base de datos, backend, frontend y pipeline) en contenedores Docker interconectados. Su función principal es desplegar PostgreSQL con almacenamiento persistente y validaciones de salud, garantizando que la API (FastAPI) y el motor de procesamiento solo arranquen cuando la base de datos esté plenamente operativa, al mismo tiempo que despliega el servidor web Nginx y mapea los volúmenes locales para permitir la sincronización del código en tiempo real sin perder información.

7.7 Backend

services

Contiene los servicios internos del sistema, responsables de ejecutar tareas clave sin interacción directa con el usuario.

- **train_predictions.py**
Gestiona el entrenamiento del modelo predictivo utilizando los datos actualizados. Prepara los datasets, entrena el modelo y genera las predicciones necesarias para su posterior consumo.
- **import_predictions.py**
Se encarga de la inserción de las predicciones generadas en la base de datos, garantizando que los resultados estén disponibles para su consulta mediante la API o herramientas analíticas.

models

Define los modelos de datos y estructuras utilizadas por el backend, facilitando la coherencia entre los datos almacenados en base de datos y los objetos utilizados por la lógica de negocio.

database.py

Módulo responsable de la **gestión de la conexión a la base de datos**. Centraliza la obtención de la cadena de conexión a PostgreSQL mediante variables de entorno, facilitando un acceso reutilizable y desacoplado desde el resto de servicios del backend.

main.py

Archivo principal de la **API backend**, implementada con **FastAPI**. Expone un endpoint que consolida información analítica y predictiva, agregando datos de ventas, predicciones, KPIs, contexto climático y riesgo de desperdicio. Actúa como punto de acceso para el dashboard y para el consumo externo de los resultados del sistema.

Dockerfile

Define el entorno de ejecución del **backend API** utilizando Docker. Configura una imagen ligera de Python, instala las dependencias necesarias y despliega el servidor web mediante **Uvicorn**, exponiendo la aplicación FastAPI de forma reproducible y preparada para su consumo externo.

7.8. Preparación del dataset para Machine Learning

[Acceso a los ficheros CSV limpios](#)

[Acceso al código del ML](#)

El entrenamiento del modelo no se realizó directamente desde la base de datos, sino a partir de **ficheros CSV**, lo que permite una mayor reproducibilidad.

7.8.1 Construcción del dataset

Para entrenar el modelo se realizaron los siguientes pasos:

1. Agregación de ventas por **fecha e id_plato**.
2. Unión de las ventas con:
 - a. datos de contexto (`dim_context`),
 - b. información del plato (`dim_menu`),
 - c. complejidad y costes de receta (`dim_recipes`).
3. Generación de nuevas variables:
 - a. número de ingredientes,
 - b. coste medio del plato.

7.8.2 Preprocesado de los datos

Dado que los modelos de Machine Learning trabajan únicamente con valores numéricos, se aplicaron técnicas de codificación:

- `LabelEncoder` para:
 - `id_plato`,
 - categoría del plato,
 - día de la semana.

Esto permitió transformar variables categóricas en valores numéricos utilizables por el modelo.

7.9. Entrenamiento del modelo de Machine Learning

Se utilizó un modelo de **RandomForestRegressor**, elegido por su robustez y su capacidad para modelar relaciones no lineales sin necesidad de normalización previa.

7.9.1 Características del modelo

- 200 árboles de decisión.
- Semilla fija para reproducibilidad.
- Uso de múltiples variables explicativas:
 - contexto temporal y meteorológico,
 - características del plato,
 - complejidad de la receta.

El modelo se entrenó con datos históricos para aprender patrones de venta.

7.9.2 Generación de predicciones futuras

Una vez entrenado el modelo, se generaron predicciones para **fechas futuras**, combinando:

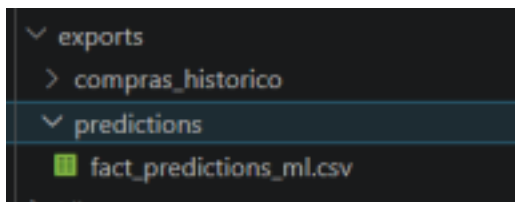
- los datos de contexto futuros,
- todos los platos disponibles.

Para cada combinación se calculó:

- la **cantidad predicha**,
- un **intervalo de confianza**, calculado a partir de la dispersión de las predicciones de los árboles del Random Forest (percentiles 10 y 90).

7.9.3. Exportación de las predicciones

Las predicciones se exportaron automáticamente a un fichero CSV:



7.9.4 Inserción automática de predicciones en PostgreSQL

Finalmente, se desarrolló un servicio en Python que:

1. Lee el CSV de predicciones.
2. Establece conexión con PostgreSQL.
3. Inserta los registros en la tabla `fact_predictions`.

La ruta del CSV se construye de forma dinámica para evitar errores de ejecución en distintos entornos.

7.9.5. Conclusión

El sistema desarrollado integra de forma completa el procesamiento de datos, el entrenamiento de modelos predictivos y la persistencia de resultados, permitiendo apoyar la toma de decisiones mediante técnicas de Big Data e Inteligencia Artificial

8. Limitaciones actuales

8.1 Bugs y problemas conocidos

El identificador de predicción (`id_prediction`) se genera de forma secuencial en el script, sin sincronización directa con la base de datos, lo que podría causar conflictos si se ejecuta el proceso varias veces sin limpiar la tabla.

El sistema no valida actualmente si una predicción ya existe en la tabla `fact_predictions`, pudiendo producir duplicados si se ejecuta el servicio más de una vez.

8.2 Atajos técnicos asumidos

No se realiza una validación cruzada formal del modelo; el entrenamiento se basa en la totalidad de los datos históricos disponibles.

El intervalo de confianza se calcula a partir de la dispersión de los árboles del Random Forest, lo cual es una aproximación válida, pero no equivale a un intervalo estadístico exacto.

8.3 Partes incompletas o no implementadas

El backend no expone todavía una API REST para consumir las predicciones desde aplicaciones externas.

No se ha desarrollado un sistema de monitorización del rendimiento del modelo para detectar degradación de la calidad de las predicciones con el tiempo.

8.4 Consideración final sobre las limitaciones

Las limitaciones descritas no impiden el correcto funcionamiento del sistema dentro del alcance del proyecto. Por el contrario, identificarlas permite establecer una base sólida para futuras mejoras y demuestra una comprensión realista de los retos asociados a sistemas de Big Data y Machine Learning en entornos reales.

9. Repositorio obligatorio

Github:

[Enlace al repositorio](#)